



Universidad Nacional Autónoma
de México

Facultad de Ciencias



Proyecto Chat

Cliente y Servidor para un Chat

Autor: Fernando Chablé Alonzo

Profesor: Canek Peláez Valdéz

Ayudantes: Cynthia Lizbeth Sánchez Urbano y Miriam Torres Bucio

Asignatura: Modelado y Programación

Fecha: 27 de septiembre de 2026

Índice

1. Resumen	2
2. Introducción	3
2.1. ¿Qué es la arquitectura Cliente Servidor?	3
2.2. ¿Qué son los sockets TCP?	3
2.3. ¿Qué es la concurrencia?	3
2.3.1. ¿Qué son los hilos de ejecución/threads?	4
2.4. ¿Qué es un protocolo?	4
3. Análisis y diseño	5
3.1. Análisis	5
3.2. Diseño	6
4. Implementación	9
5. Conclusiones	10
5.1. ¿Qué aprendí?	10
5.2. ¿Qué se me dificultó?	10
5.3. ¿Qué pude mejorar?	10
5.4. Si quisiera expandir el sistema, ¿Qué propondría?	10
6. Bibliografía	11

1. Resumen

En este reporte se presenta el diseño e implementación de un sistema de chat concurrente basado en la arquitectura cliente-servidor.

El sistema se fundamenta en el uso de sockets TCP y la gestión de hilos de ejecución (threads) para garantizar una comunicación fluida, ordenada y simultánea entre múltiples clientes. Para la construcción del proyecto, el servidor fue desarrollado en lenguaje C haciendo uso de las librería cJSON para un control eficiente de memoria, mientras que el cliente se estructuró en C# mediante el SDK de .NET. La arquitectura modular sigue el patrón Modelo-Vista-Controlador (MVC) y cumple con un protocolo de comunicación de doce comandos que abarca la identificación de usuarios, mensajería pública y privada, y la administración dinámica de salas apoyada en listas ligadas simples.

Finalmente, se discuten los aprendizajes adquiridos, las dificultades técnicas durante la implementación en red y las posibles líneas de trabajo futuro.

(No se me ocurre más resumen xd).

2. Introducción

Este proyecto contiene algunos conceptos medianamente complejos, tales como **sockets TCP**, **concurrency**, **hilos de ejecución**, entre algunos otros por lo que veo necesario explicarlos brevemente a continuación:

2.1. ¿Qué es la arquitectura Cliente Servidor?

[1] La **arquitectura Cliente Servidor** podemos definirla como un modelo de diseño de software en el cual las aplicaciones se dividen el trabajo entre dos componentes principales: **clientes** y **servidores**. Esta separación facilita la gestión de recursos y hace que ambos interactúen de manera eficiente, pues el cliente se dedica a realizar solicitudes de *servicios* o *datos* al servidor, y el servidor se dedica a proporcionar esos *recursos* que requiere el cliente.



Figura 1: Diagrama de la arquitectura cliente-servidor. Fuente: [2]

En el caso de este proyecto, el **servidor** se encarga de escuchar todas solicitudes de los diferentes **clientes**, sean solicitudes para *identificarse*, *mandar mensajes*, *crear salas*, entre otras. El **cliente** le manda una solicitud al **servidor** con datos que proporcionó el usuario, el **servidor** procesa esos datos y le responde al **cliente** (o incluso a los demás clientes) su solicitud para que el usuario los vea.

2.2. ¿Qué son los sockets TCP?

[3] Un **socket** lo podemos definir como un punto final de comunicación entre dos programas o dispositivos, son los responsables de conectar esos mismos dos programas o dispositivos. Los **sockets** siguen dos protocolos diferentes: **TCP** y **UDP**, y en el caso de este proyecto se usaron **sockets TCP**. El protocolo **TCP** es un protocolo de la capa de transporte que está orientado a **conexión**, por lo que antes de intercambiar datos debe haber un paso para poder establecer una comunicación. Además, **TCP** garantiza que la información llegue en correcto estado, pues si la información tarda en llegar o llega dañada, el propio **TCP** se encarga de volverla a mandar, y una última ventaja que tienen es que los datos que manda se envían en **orden**.

2.3. ¿Qué es la concurrencia?

[4] La **concurrencia** podemos definirla como la **ejecución simultanea** de múltiples procesos, esto permite que diferentes partes de un programa funcionen de manera independiente, lo que eficientiza el uso de recursos y la experiencia del usuario. Dentro de la **concurrencia** tenemos otro concepto importante, que son los **hilos de ejecución**.

2.3.1. ¿Qué son los hilos de ejecución/threads?

Los **hilos de ejecución** (mejor conocidos como **threads**) son por así decirlo caminos de ejecución pequeños dentro de un proceso. En un entorno concurrente, muchos **hilos de ejecución** pueden coexistir, donde cada uno tiene una tarea específica.

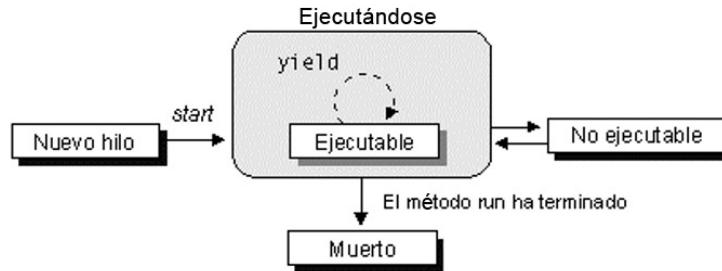


Figura 2: Diagrama de un hilo de ejecución. Fuente: [5]

2.4. ¿Qué es un protocolo?

Por último, un **protocolo de comunicación** podemos definirlo como las reglas o el formato que tanto el **cliente** como el **servidor** siguen para entenderse sin que se presente ninguna ambigüedad o algo parecido. (Sí, no hay mucho que decir de qué es un protocolo hasta donde yo sé).

3. Análisis y diseño

En esta sección vamos a ver (de manera un poco breve) el análisis que hice sobre nuestro problema y el diseño de la solución que hice. En este caso, nuestro problema era hacer un chat usando la arquitectura Cliente Servidor, que fuera concurrente y, lo más importante, **FUNCIONAL**.

3.1. Análisis

Pude deducir rápidamente que el problema sí era computable, pues si dividimos el problema en diferentes subproblemas nos darémos cuenta que sí se pueden hacer en un número finito de pasos, más considerando que la cantidad de usuarios que se conectarán al servidor son finitos, por lo que (mientras esté bien hecho), todos esos subproblemas tendrán un fin. Los diferentes subproblemas en los que lo dividí son:

- **Servidor**

- Crear y manejar los hilos de ejecución para cada usuario.
- Crear un socket para poder escuchar y aceptar conexiones entrantes.
- Procesar y responder los datos que mande cada usuario.

- **Cliente**

- Crear un socket para poder conectarse al servidor.
- Procesar y mandar los datos del usuario al servidor.

Además, gracias al protocolo que nos proporcionó el profesor, ya teníamos otra parte del problema (todo lo que constituía al chat) dividida en más subproblemas:

- **IDENTIFY**: El usuario se identifica con un username único. Si no se identifica no puede hacer nada.
- **STATUS**: El usuario cambia su estado actual a otro.
- **USERS**: Se le muestra al usuario una lista con todos los usuarios conectados.
- **TEXT**: El usuario manda un texto privado a otro.
- **PUBLIC TEXT**: El usuario manda un texto público.
- **NEW ROOM**: El usuario crea una nueva sala.
- **INVITE**: El usuario invita a más usuarios a una sala.
- **JOIN ROOM**: El usuario se une a una sala en la que fue invitado.
- **ROOM USERS**: Se le muestra al usuario una lista con los usuarios de una sala.
- **ROOM TEXT**: El usuario manda un mensaje a una sala.
- **LEAVE ROOM**: El usuario sale de una sala.
- **DISCONNECT**: El usuario se desconecta del chat.

Y sí, como ya se pudo dar cuenta querido lector, el chat también tendrá **salas** (como los grupos de WhatsApp o los servidores de Discord).

3.2. Diseño

Tuve que definir unas cuantas estructuras para poder manejar a los *usuarios* y a las *salas*, y a su vez definí dos listas para poder manejar a todos los *usuarios* y a todas las *salas*.

■ **ClienteNodo:**

- *Atributos:*
 - Socket (un entero que represente al socket del usuario).
 - Username (un string que represente el username del usuario).
 - Status (un string que represente el estado actual del usuario).
 - Siguiente (un puntero que apunte a otro ClienteNodo que represente al usuario que sigue del actual).

■ **ListaClientes:**

- *Atributos:*
 - Cabeza (un puntero que apunte a un ClienteNodo que está en la lista y que representa a la cabeza de la misma).
 - NumClientes (un entero que represente a la cantidad de usuarios que hay en la lista).
- *Métodos:*
 - `crear_lista`: Inicializa una lista de clientes desde cero.
 - `insertar_cliente`: Crea un nuevo cliente con los datos proporcionados y lo mete a la lista.
 - `eliminar_cliente`: Elimina a un cliente de la lista basándose en el socket proporcionado.
 - `buscar_cliente`: Revisa si un cliente existe en la lista.
 - `limpiar_lista`: Se encarga de limpiar la memoria usada por la lista.

■ **NodoSala:**

- *Atributos:*
 - Room_name (un string que represente el nombre de la sala).
 - Usuarios (un puntero que apunta a una ListaClientes que almacena a los usuarios de la sala).
 - Invitados (un puntero que apunta a otra ListaClientes que almacena a los usuarios que se han invitado a la sala).
 - Siguiente (un puntero que apunta a un NodoSala que represente a la sala que sigue de la actual).

■ **ListaSalas:**

- *Atributos:*
 - Cabeza (un puntero que apunta a un NodoSala que está en la lista y que representa a la cabeza de la misma).
 - Num_salas (un entero que represente a la cantidad de salas que hay en la lista).
- *Métodos:*
 - `crear_lista_salas`: Inicializa una lista de salas desde cero.
 - `insertar_sala`: Crea una nueva sala con el nombre proporcionado y lo mete a la lista.
 - `eliminar_sala`: Elimina una sala de la lista basándose en el nombre proporcionado.
 - `buscar_sala`: Revisa si una sala existe en la lista.
 - `obtener_sala`: Obtiene la dirección de la sala en la lista y del nombre proporcionado.
 - `limpiar_lista_salas`: Se encarga de limpiar la memoria usada por la lista.

Como se puede ver, elegí implementar listas ligadas simples para poder almacenar a los usuarios y las salas, esto porque me parecen bastante fáciles de implementar y eficientes, pues tienen un tamaño dinámico y son fáciles de manejar.

El protocolo proporcionado por el profesor incluye **doce métodos**, métodos que salen en el **análisis**, y para explicar cómo los diseñé usaré diagramas secuenciales en UML, pero para no inundar el reporte con doce diagramas, los voy a resumir a tres.

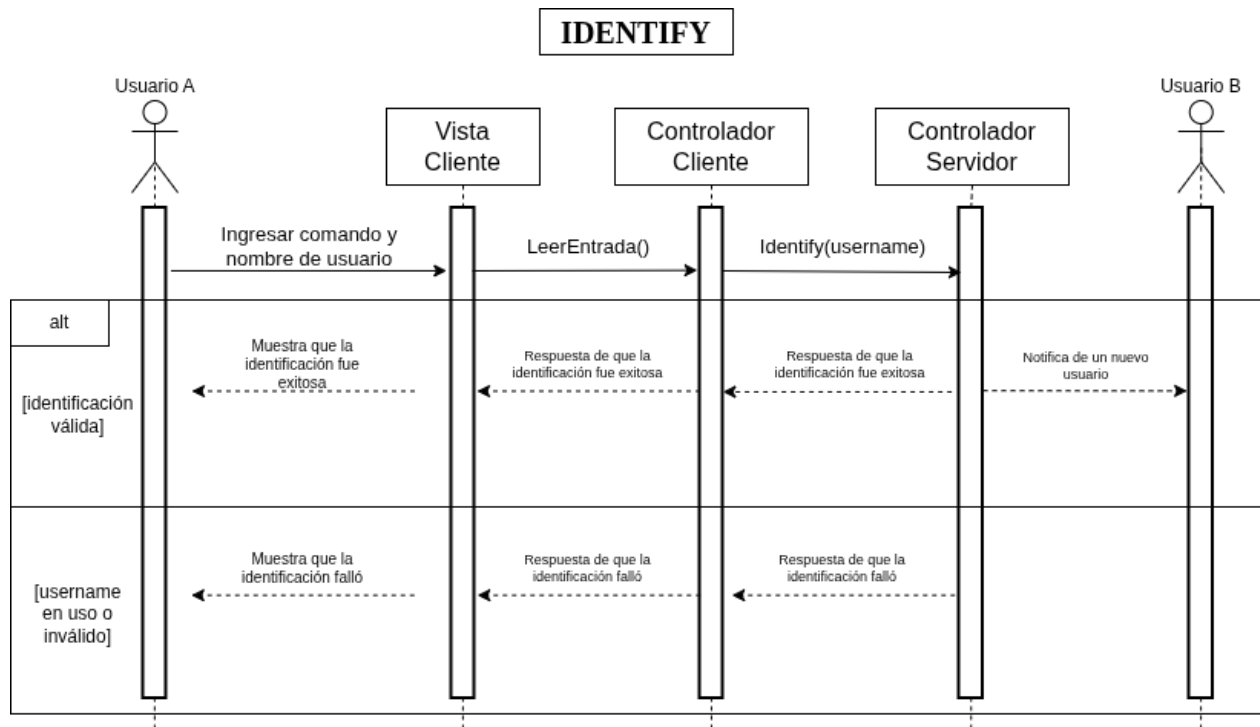


Figura 3: Diagrama UML de IDENTIFY.

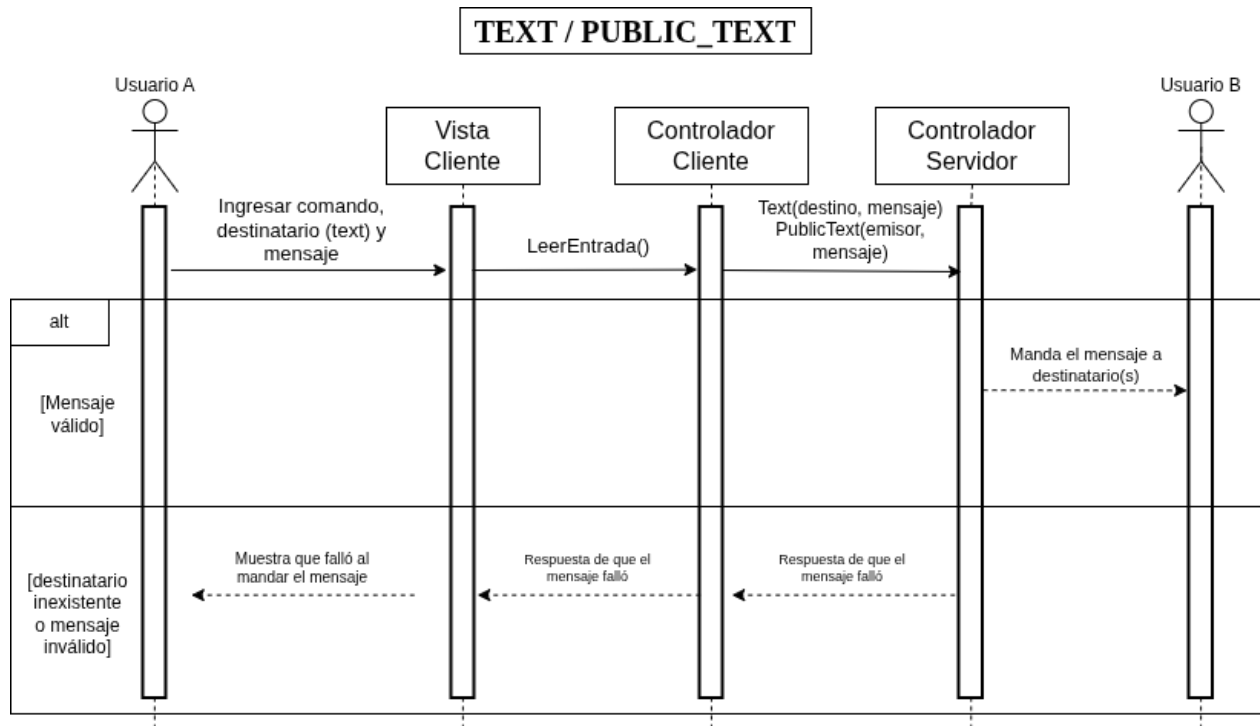


Figura 4: Diagrama UML de TEXT y PUBLIC_TEXT.

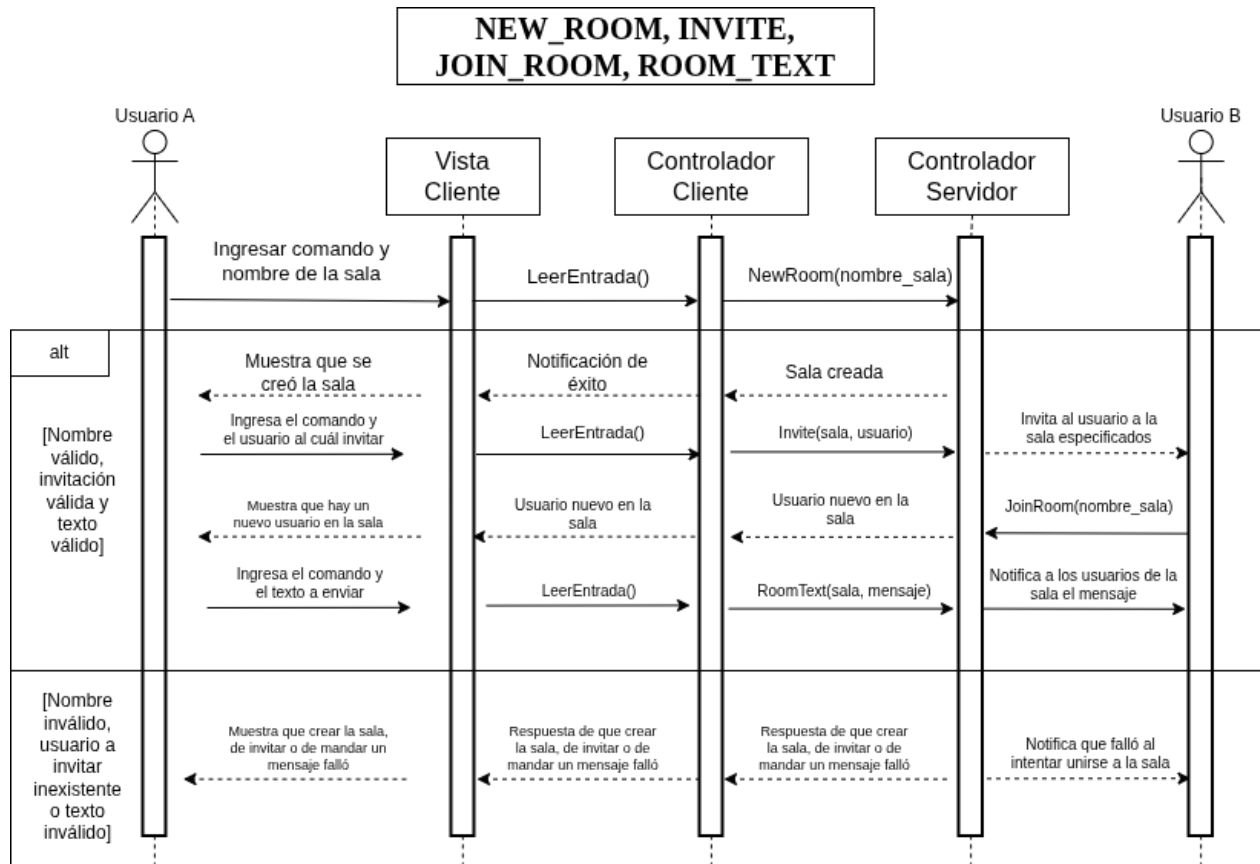


Figura 5: Diagrama UML de NEW_ROOM, INVITE, JOIN_ROOM y ROOM_TEXT.

No veo necesario hacer los diagramas UML de los demás métodos pues me parecen demasiado sencillos y saturarán de información el reporte.

Sinceramente no se me ocurre una mejor manera para poder implementar estos métodos, pues estas implementaciones cubren prácticamente todos los casos posibles y tienen una respuesta para cada uno.

4. Implementación

Para poder implementar el diseño que hice, usé dos lenguajes de programación: **C** para hacer el servidor y **C#** para hacer el cliente. Usé **C** para el servidor pues me daba curiosidad aprender este lenguaje, más considerando los punteros, y por eso mismo me parece un lenguaje bastante eficiente (siempre y cuando manejes bien la memoria, claro); y usé **C#** para el cliente pues era el lenguaje del que más tenía conocimiento (después de Java, pero ese no lo podíamos usar).

Los únicos requerimientos para poder compilar y ejecutar el servidor y el cliente son los siguientes:

■ Servidor:

- Para poder compilar el servidor se necesita tener instalado el compilador **GCC** (hasta donde sé, todas las distribuciones de Linux ya lo tienen por default), realmente no se necesita algo más, solo tendrías que ubicarte en la carpeta de nombre **servidor**, y ejecutar el siguiente comando: **gcc -Wall main.c modelo/ListaClientes.c modelo/ListaSalas.c vista/red_vista.c controlador/chat_controlador.c -lcjson -lpthread -o servidor** Luego, para poder ejecutar el servidor, solamente tienes que escribir **./servidor** en la carpeta donde se creó el ejecutable.

■ Cliente:

- Para poder compilar el cliente se necesita tener instalado el **SDK** de **.NET**, pues contiene el compilador de **C#**, el runtime de **.NET** y las librerías base de **C#**. Para compilar y ejecutar el cliente, tendrías que ubicarte en la carpeta de nombre **ClienteChat** y ejecutar el siguiente comando: **dotnet run**, con ese simple comando se compila todo el proyecto de **C#** y lo ejecuta automáticamente.

5. Conclusiones

Me parece que las conclusiones las puedo dividir en varias secciones.

5.1. ¿Qué aprendí?

En lo personal yo aprendí a manejar sockets, a construir un servidor y un cliente usando la arquitectura cliente servidor, a manejar varios casos que pueden hacer los usuarios, a poder dividir un proyecto así de grande (sé que no es grande pero es el más grande que he hecho hasta la fecha) usando el patrón MVC, a poder seguir un protocolo sin perderme en el intento, y a usar mejor C y C#.

5.2. ¿Qué se me dificultó?

Pienso que lo más difícil fue entender el cómo hacer el servidor, cómo hacer el cliente y cómo hacer que se comuniquen, esto porque no tenía absolutamente nada de conocimiento sobre sockets ni de las librerías que los implementan en C y C#. A su vez, tampoco sabía nada sobre la arquitectura cliente servidor, y tampoco sobre hilos de ejecución, por lo que aprender de estos conceptos fue lo que más me robó tiempo considerando que es de lo más rápido (porque obviamente implementar el protocolo fue lo que más me robo tiempo, pero eso fue por ser muy tardado).

5.3. ¿Qué pude mejorar?

Sobre todo creo que puedo mejorar en cómo estructuro mi código, pues en algunos métodos se repiten las mismas ocho líneas de código, cosa que se podía solucionar haciendo un método auxiliar con esas ocho líneas, además que tanto código puede llegar a abrumar al que lo intenta comprender a falta de comentarios. Otra cosa que también pude haber mejorado es haber implementado una interfaz gráfica, pues es mucho más intuitivo usar la interfaz gráfica de un programa que usar comandos en la terminal.

5.4. Si quisiera expandir el sistema, ¿Qué propondría?

En mi caso, yo propondría que el chat tenga soporte para mandar diferentes archivos, sean archivos de texto, de audio, imágenes, etc. Además, no estaría mal poner alguna clase de filtro en el chat, pues como está hecho ahorita cualquier cosa (incluyendo ilícitas) pueden expresarse en el chat. Y por último, intentaría hacer que el IDENTIFY sea un poco más estricto, y en vez de identificarte solo con el nombre, te tengas que identificar con tu nombre y alguna contraseña que hayas hecho antes (para evitar robos de identidad o algo parecido).

6. Bibliografía

Referencias

- [1] Gomez, C. (2024). *Qué es la arquitectura cliente servidor y cómo funciona*. <https://www.daemon4.com/empresa/noticias/arquitectura-cliente-servidor/>
- [2] Invarato, R. (2019). *Cliente vs Servidor*. <https://jarroba.com/cliente-vs-servidor/>
- [3] De Luz, S. (2025). *Qué es un socket TCP o UDP y qué diferencias hay con los puertos*. <https://www.redeszone.net/tutoriales/configuracion-puertos/que-es-socket-tcp-udp-diferencias-puertos/>
- [4] Lenovo (s/f). *¿Qué es la concurrencia?*. <https://www.lenovo.com/mx/es/glosario/concurrencia/>
- [5] UNAM (s/f). *Hilos*. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3084/mod_resource/content/1/UAPA-Hilos/index.html